

WT7 Application Description

Generated from WT7_APP_DESCRIPTION.md

WT7 is a Linux radio astronomy antenna-control application for a two-antenna interferometry station. It replaces the day-to-day control role previously provided by WinTrak, while retaining compatibility with the existing Arduino controller firmware and the decoded WinTrak serial command protocol.

The application controls two independent antenna systems, normally labelled East and West. Each antenna has its own USB serial controller, slew-drive system, quadrature encoder position feedback, and local OLED display. WT7 provides guarded manual motion, Sun/Moon/source tracking, parking, calibration, scan calibration, Y factor measurements, event logging, and B210 SDR power measurement support.

High-Level Overview

What The App Does

WT7 is an operator interface and control system for pointing two radio astronomy antennas safely and repeatably.

At a high level, it:

- Connects to two Arduino-based antenna controllers over USB serial.
- Reads raw azimuth and elevation encoder positions from each antenna.
- Applies calibration offsets to produce displayed antenna AZ/EL.
- Commands azimuth and elevation slew drives for manual movement, tracking, parking, scan calibration, peak calibration, and Y factor measurements.
- Tracks the Sun, the Moon, or user-defined celestial radio sources.
- Calculates live Sun, Moon, and user-source positions from observer location and time.
- Enforces software azimuth and elevation limits because the installation does not currently have physical limit switches.
- Updates the local OLED displays mounted on each controller.
- Measures received power using a dual-channel Ettus B210 SDR.
- Uses received power during scan calibration and Y factor workflows.
- Logs important operating events and measurement outputs for later diagnosis.

The application is intended to become the primary station-control program for normal radio astronomy observing and calibration work.

How The App Does It

WT7 separates the station-control problem into several layers:

- A PyQt5 GUI collects user commands and displays system state.
- Configuration loaders read and save the INI file containing observer, antenna, source, tracking, calibration, scan, Y factor, and B210 settings.
- Astronomy functions calculate apparent AZ/EL and hour angle for sources.
- A decoded WinTrak protocol layer communicates with the Arduino controllers.
- A safety layer validates every target and guarded movement before and during drive motion.
- Antenna-session objects own serial I/O, calibrated positions, guarded slews, OLED updates, and per-antenna locks.

- Tracking, parking, scan calibration, Y factor, and peak calibration workflows run as background operations so the GUI remains responsive.
- A B210 power-meter layer opens a synchronized two-channel receive stream and converts blocks of I/Q samples into channel power readings.
- Logs and CSV measurement files preserve what happened without overloading the GUI with diagnostic detail.

The app does not directly control the SVH3 drive electronics. Instead, it sends the Arduino controller the same style of low-level commands that WinTrak sends. The Arduino then controls the drive channels, reads quadrature encoder counts, maintains the absolute position used by the controller, and drives the OLED.

Detailed Functional Description

Main Operating Surface

The WT7 GUI is built around a compact station dashboard. It shows:

- Current selected target.
- Target AZ, EL, and hour angle.
- Live Sun and Moon AZ/EL.
- Local time, UTC, and LMST.
- East and West antenna cards.
- B210 dual-channel power readings.
- Recent important events.

Each antenna card shows:

- Calibrated AZ and EL.
- AZ and EL pointing error relative to the current target.
- Limit status.
- Current operating mode.
- Current target.
- Manual jog controls in cross formation.
- Per-antenna status, such as disconnected, stopped, tracking, slewing, parking, scan, Y factor, or fault.

The GUI is intended to use fixed-width value fields so that changing live numbers do not resize the display or clip neighbouring fields.

Configuration

WT7 is configured from an INI file, normally:

```
python3 wt7_ubuntu_gui.py --config wt7_ubuntu.ini
```

The configuration includes:

- Observer latitude and longitude.
- Selected source.
- Tracking interval and tolerances.
- Slow-speed tracking parameters.

- Event-log retention.
- Per-antenna USB serial port and baud rate.
- Per-antenna open delay.
- Per-antenna manual and tracking speeds.
- Per-antenna azimuth low-to-high compensation.
- Per-antenna calibration offsets.
- Per-antenna azimuth and elevation limits.
- Per-antenna park positions.
- User source list.
- Scan calibration parameters.
- Y factor parameters.
- B210 frequency, sample rate, bandwidth, gains, clock source, and channel mapping.

Most operational settings are editable from the GUI and saved back to the INI file.

Antenna Controller Communication

Each antenna is controlled through a USB serial port using the decoded WinTrak Arduino protocol. The normal baud rate is 9600.

The controller layer can:

- Read azimuth position.
- Read elevation position.
- Start azimuth clockwise or counter-clockwise movement.
- Start elevation up or down movement.
- Stop azimuth.
- Stop elevation.
- Stop all axes.
- Read encoder information.
- Write OLED text.
- Update OLED status pages.

Each SafeAntenna session serializes controller access with a lock, so command and stop traffic do not corrupt serial transactions.

Position And Calibration Model

The encoder hardware is a quadrature pulse generator. The encoder itself is not written to. The Arduino/controller maintains the absolute position derived from encoder counts.

WT7 treats positions as:

- Raw AZ/EL: values reported by the controller.
- Calibrated AZ/EL: raw values plus WT7 calibration offsets.

Main GUI antenna positions are calibrated positions. The calibration dialog shows raw positions and editable AZ/EL offsets. Calibration changes are shared between the normal calibration and peak calibration workflows.

Safety Model

Software safety limits are a core requirement because the antenna systems do not currently have physical limit switches.

WT7 validates:

- Target azimuth is inside the allowed azimuth arc.
- Target elevation is inside configured elevation limits.
- Elevation remains within a physically sensible antenna range.
- Movement direction does not approach a configured software margin.
- The azimuth path does not cross the forbidden dead-zone.
- Park positions are legal before motion.
- Source targets below or outside limits are rejected before drive movement.

The azimuth model supports wrap-around limits. For example:

```
az_min = 270  
az_max = 265
```

means the allowed region is from 270 through 360 and from 0 through 265. The small region from 265 to 270 is forbidden. WT7 must never drive through that dead-zone.

Every guarded movement repeatedly:

- Reads current position.
- Checks limits.
- Chooses the permitted direction to the target.
- Starts the required axis or axes.
- Polls position during movement.
- Applies stop events.
- Checks for no-progress or timeout conditions.
- Stops the axis at the end of the move or on fault.

Stop commands must take priority over tracking, scans, Y factor measurements, parking, and calibration workflows.

Tracking

WT7 can track:

- Sun.
- Moon.
- A selected user source.

Tracking uses the observer location and current time to calculate live AZ/EL. User sources are defined by:

- Name.
- Right ascension.
- Declination.
- 4800 MHz flux.

Tracking has independent AZ and EL parameters:

- Start tolerance.
- Stop tolerance.
- Tracking speed.
- Slow speed.
- Slow-speed threshold.

The start tolerance defines how far from the nominal target the antenna can be before a correction starts. The stop tolerance defines how close to the nominal target the antenna should be before a correction stops. Negative stop tolerance is supported to allow deliberate lead/overshoot behaviour when needed.

Gross moves between targets are shown as slewing. Once the antenna has acquired the target, small periodic corrections should be shown as tracking rather than slewing.

Azimuth Hysteresis Compensation

The azimuth drives have measurable backlash/hysteresis. WT7 supports a per-antenna low-to-high azimuth compensation value.

The compensation is applied when an azimuth tracking or slew command requires a low-to-high azimuth movement. The intended effect is to bias the commanded target to account for the different pointing result produced by changing drive direction.

For scan calibration, WT7 also supports scanning in either high-to-low or low-to-high azimuth direction so backlash can be measured. The normal default is high-to-low.

Manual Jogging

Manual jogging is guarded:

- The operator holds a direction button to drive.
- Releasing the button stops the axis.
- A maximum held jog time prevents indefinite drive commands.
- Position and limit checks continue while the jog is active.
- Manual speed is configurable per antenna.

Manual controls are disabled or guarded when the antenna is not connected.

Parking

The Park command drives each antenna to its configured park AZ/EL position.

Parking requirements:

- Park targets are validated before motion.
- Each antenna slews to its own park position.
- Parking status is displayed.
- On successful park, the app should stop and disconnect.
- If one antenna faults during park, the condition must be visible and logged.

OLED Displays

Each Arduino controller drives a local OLED display. WT7 writes display text using the decoded controller protocol.

The OLED should show:

- Configured antenna label, such as East or West.
- Calibrated AZ and EL.
- Current target source.
- Target AZ and EL when relevant.
- State such as stopped, tracking, slewing, scan, Y factor, park, fault, or offline.

The OLED should be populated immediately after connection and should not show a selected scan or Y factor state on the non-selected antenna.

B210 Power Meter

WT7 uses an Ettus B210 SDR as a dual-channel power meter.

The B210 power layer:

- Opens a SoapySDR/UHD B210 device.
- Configures both receive channels.
- Sets frequency, sample rate, RF bandwidth, gains, and clock source.
- Disables gain mode/AGC for manual gain operation.
- Activates a time-aligned two-channel receive stream.
- Reads complex I/Q sample blocks from both channels.
- Calculates mean I/Q power for each channel.
- Displays channel A and channel B power in dBFS or calibrated dBm.

The number of I/Q samples per B210 power update is normally:

$$\text{sample_rate_hz} / \text{GUI_update_rate_hz}$$

The displayed average is a smoothing average over the configured number of power readings, not merely that number of raw I/Q samples.

The SDR can be released so other applications can access it. When released, old readings should be cleared so stale levels are not mistaken for live data.

B210 Calibration

B210 calibration is intended to convert dBFS readings to dBm using a signal generator.

Calibration requirements:

- Calibration is associated with frequency, sample rate, bandwidth, gain A, gain B, and channel.
- Calibration is captured one channel at a time.
- Calibration range is from -40 dBm down to -110 dBm in 10 dB steps.
- Interpolation and extrapolation are used as necessary.
- If frequency, sample rate, bandwidth, gain, or channel does not match the stored calibration, the power display must be flagged uncalibrated.

Scan Calibration

Scan calibration sweeps one selected antenna through a source and measures power at each offset.

The scan calibration dialog includes:

- Antenna selection.
- Span in degrees.
- Increment in degrees.
- Dwell time per point.
- Number of repeated scans.
- AZ scan high-to-low checkbox.
- AZ scan button.
- EL scan button.
- Stop scan button.

During a scan:

- The selected antenna is offset from the nominal tracked source position.
- The other antenna should not falsely show scan activity.
- The app continues to track the source while applying the scan offset.
- Power is collected during the dwell time.
- Data is saved as CSV under scan/.
- The scan result plot is displayed immediately after the last measurement, without waiting for the antenna to return to nominal tracking.
- The plot window must be closeable without closing the Scan Cal dialog.
- Stop Scan must stop the active scan antenna, clear scan offsets, and leave the app ready for another scan or tracking operation.

Plots include:

- Data points.
- Graticule.
- Boresight line at zero offset.
- Gaussian fit with sloped baseline.
- Boresight error.
- FWHM.
- Peak level.
- Fit residual/RMS.

Peak Calibration

Peak calibration supports a faster operator-assisted workflow for peaking on a source.

The operator should be able to:

- Select Sun, Moon, or user source.
- Select East or West antenna.

- Track one axis while manually jogging the other.
- Peak source power manually.
- Lock AZ calibration or EL calibration with a single button.

This workflow is intended to avoid delays between reaching peak signal and applying calibration.

Y Factor Measurement

The Y factor workflow compares hot-source power to cold-sky power.

Hot target options:

- Sun.
- Moon.
- User source.

Cold target options:

- Sun AZ / EL 80.
- Moon AZ / EL 80.
- Manual AZ/EL.
- Manual RA/Dec.

The user can configure:

- Selected antenna.
- Number of measurements.
- Dwell time.
- Hot/cold workflow order.

The alternate workflow is intended to reduce elapsed time by allowing measurements to proceed hot-to-cold then cold-to-hot rather than always returning to hot before each pair.

Y factor results should:

- Use B210 power measurements.
- Average power over the dwell time.
- Report final Y factor in dB to one decimal place.
- Save each measurement cycle to CSV under `yfactor/`.
- Include timestamp, antenna, source, hot/cold target coordinates, power readings, and Y factor.

Source Management

The source dialog maintains user-defined radio sources.

It should:

- List source name, RA, Dec, current AZ, current EL, and 4800 MHz flux.
- Continuously update current AZ/EL values while open.
- Allow adding rows.
- Allow deleting rows.

- Allow selecting the active source without a redundant selection workflow.
- Save the source table back to the INI file.

Event Logging

WT7 keeps an event log for fault finding.

The event log should capture important station and app events without growing too quickly. Logging should focus on significant operational events and fault diagnostics rather than every routine poll.

Events include:

- App start and stop.
- Connect and disconnect.
- Tracking start and stop.
- Slew start and stop.
- Parking start and stop.
- Scan start, stop, completion, and fault.
- Y factor start, stop, completion, and fault.
- B210 start, stop, and fault.
- Calibration changes.
- Safety stops.
- Controller communication failures.
- Motion anomalies.

Retention days are configurable.

Measurement Logging

Measurement files are separate from the event log.

Current measurement outputs include:

- Scan calibration CSV files under `scan/`.
- Y factor CSV files under `yfactor/`.
- B210 power logs when explicitly started by the operator.

Measurement logs should include enough metadata to be useful after the observation.

Requirements

This section consolidates the requirements discussed during development.

System And Hardware Requirements

- The app shall run on Linux.
- The app shall support Raspberry Pi OS and Ubuntu.
- The app shall communicate with two Arduino-based antenna controllers over USB serial.

- The app shall use persistent `/dev/serial/by-id/...` paths where possible so East and West controllers do not swap.
- The app shall support two independent antennas used for interferometry.
- The app shall support controller OLED display updates.
- The app shall support quadrature encoder systems whose absolute position is maintained by the controller.
- The app shall not require physical limit switches to operate safely.
- The app shall support an Ettus B210 SDR for two-channel power measurement.
- The app shall release the B210 when requested so other applications can use it.

Safety Requirements

- Software limits are mandatory and shall protect the antenna systems from overrun damage.
- Elevation movement shall always be constrained to the configured EL limits.
- Elevation shall be treated as physically limited to the 0 to 90 degree range, further constrained by configured limits.
- Azimuth shall support wrap-around allowed regions and forbidden dead-zones.
- The app shall never intentionally drive through the configured azimuth dead-zone.
- All commanded target positions shall be checked before movement.
- All movement shall be checked while in progress.
- Manual jogs shall have configurable maximum held time.
- Stop commands shall override all other operations.
- The app shall detect no-progress movement conditions where commanded motion does not produce encoder change.
- The app shall detect position jumps that suggest invalid or inconsistent encoder data.
- The app shall flag unexpected antenna movement when no command is active.
- The app shall flag movement in the wrong direction.
- The app shall log safety-related failures clearly.
- If the app cannot correct a hazardous situation, it shall alert the operator so external action such as removing power can be taken.

Connection Requirements

- East and West controller connection attempts shall be independent so a failed controller does not make the other controller slow or unreliable.
- Connection shall handle controller open delay.
- The app shall indicate disconnected, connecting, connected, stopped, tracking, slewing, parking, scan, Y factor, offline, and fault states.
- The app shall populate OLED displays immediately after successful connection.
- The app shall clear antenna position values after disconnection rather than leaving stale values displayed.

Tracking Requirements

- The app shall track the Sun.
- The app shall track the Moon.
- The app shall track a selected user source.
- The app shall continuously calculate and display Sun and Moon positions.
- The app shall show hour angle for Sun, Moon, and user sources.
- The app shall allow tracking interval to be set from the GUI.

- The app shall allow independent AZ and EL start tolerances.
- The app shall allow independent AZ and EL stop tolerances.
- The app shall allow negative stop tolerance where deliberate lead/overshoot is required.
- The app shall allow independent AZ and EL tracking speeds.
- The app shall allow independent AZ and EL slow speeds.
- The app shall allow independent AZ and EL slow-speed thresholds.
- The app shall drive both axes concurrently where required.
- The app shall not show gross slewing once the antenna has reached the source and is only making normal tracking corrections.
- Changing tracking or configuration dialogs shall not freeze tracking.
- Changing from one tracking target to another shall transition cleanly.
- A failed or invalid target command shall not leave a cyclic old/new tracking command loop running.

Azimuth Path And Hysteresis Requirements

- The app shall choose the allowed azimuth path, not merely shortest geometric path, when an azimuth dead-zone exists.
- The app shall handle tracking across 0 degrees without hunting.
- The app shall include per-antenna azimuth low-to-high compensation.
- The app shall apply low-to-high compensation during tracking and slewing when the commanded azimuth movement is low-to-high.
- The app shall allow azimuth scan direction to be selected for backlash assessment.
- The default azimuth scan direction shall be high-to-low.
- During scan calibration, low-to-high compensation shall not distort the deliberate backlash measurement.
- Scan preload/backlash take-up shall use measured backlash values where appropriate.

Manual Control Requirements

- Manual controls shall move while a button is held and stop on release.
- Manual controls shall use configurable speed.
- Manual controls shall stop if max held jog time is exceeded.
- Manual control errors shall not latch unnecessarily.
- Manual controls shall not interfere with ongoing autonomous slews merely because the mouse cursor passes over a button.
- Buttons should provide visible feedback when pressed.

Parking Requirements

- The app shall include a Park command.
- Park positions shall be configurable from the GUI.
- Park shall drive both antennas to predefined positions.
- Park shall stop and disconnect the app after successful completion.
- If one antenna fails during park, the fault shall be visible and logged.

Calibration Requirements

- Main GUI antenna positions shall display calibrated AZ/EL only.
- Raw AZ/EL shall be shown in the calibration menu.
- Calibration offsets shall be visible in the calibration menu.
- Calibration offsets shall be directly editable.
- Calibration offsets shall be shared consistently between standard calibration and peak calibration.
- The calibration workflow shall support calibrating one axis at a time.
- The calibration workflow shall allow one axis to track while the other axis is manually peaked.
- Calibration from target shall use the current displayed Sun, Moon, or source AZ/EL as the actual pointing reference.
- Peak calibration shall allow one-button lock of AZ or EL calibration after peaking signal.
- Raw values in peak calibration shall be true raw values, not calibrated values.

Scan Calibration Requirements

- The app shall scan one selected antenna at a time.
- The app shall support AZ scans.
- The app shall support EL scans.
- Scan parameters shall include span, increment, dwell time, scan count, antenna selection, and AZ scan direction.
- Scan parameters shall be saved to the INI file.
- The app shall keep tracking the source while scan offsets are applied.
- Scan data shall include timestamp, offset, nominal target, actual target, antenna position, raw position, power value, power unit, and channel.
- Scan data shall be saved under the scan/ directory.
- Scan plots shall include graticule, boresight line, data trace, Gaussian fit, and fitted boresight error.
- Scan result plot windows shall be closeable without closing the Scan Cal dialog.
- Scan plots shall be displayed immediately after the last measurement.
- Stop Scan shall stop the selected antenna promptly.
- Stop Scan shall clear the scan offset.
- Stop Scan shall leave the app ready for another scan or other action.

Y Factor Requirements

- The app shall provide a Y Factor dialog.
- The app shall support Sun, Moon, and selected user source as hot targets.
- The app shall support Sun AZ / EL 80 and Moon AZ / EL 80 cold targets.
- The app shall support user-defined cold AZ/EL.
- The app shall support user-defined cold RA/Dec.
- The app shall stop the non-selected antenna during Y factor measurement.
- The app shall track the selected antenna correctly between hot and cold positions.
- The app shall support multiple Y factor measurements.
- The app shall support both repeated hot-cold pairs and alternating hot/cold order.
- The default Y factor workflow shall minimise unnecessary return slews.
- Dwell time shall be used to collect and average power.
- The final displayed Y factor shall be in dB only, to one decimal place.
- Y factor logs shall be written under yfactor/.

- Y factor shall not leave stale or cyclic tracking state after stop or fault.

B210 Requirements

- The app shall support dual-channel B210 power measurements.
- The app shall map East and West antennas to configured B210 channels.
- The app shall allow B210 power to be switched on and released.
- The app shall configure centre frequency in MHz.
- The app shall configure sample rate in ksps.
- The app shall configure measurement bandwidth in kHz.
- The app shall configure gain A and gain B independently.
- The app shall configure clock source.
- The app shall configure GUI update rate.
- The app shall configure smoothing average.
- The app shall disable AGC/gain mode for calibrated measurements.
- The app shall flag uncalibrated B210 readings when no matching calibration is loaded.
- The app shall clear displayed power when the SDR is released.
- The app shall handle B210 overflow and timeout faults clearly.
- The app shall avoid trying to use the old RTL-SDR path for B210 functions.
- The app shall support one-channel-at-a-time B210 calibration.
- B210 logs shall clearly indicate when logging starts and stops.

Source Requirements

- The app shall store user sources in the INI file.
- Each source shall include name, RA, Dec, and 4800 MHz flux.
- The source menu shall show current AZ/EL for each source.
- Source positions shall update continuously while the source menu is open.
- The source menu shall allow adding and deleting sources.
- The source menu shall allow selecting the active tracking source.
- Source tables shall provide scroll bars when the number of sources exceeds the visible area.

OLED Requirements

- OLED displays shall show configured antenna labels.
- OLED displays shall show calibrated AZ and EL.
- OLED displays shall show current source/target context.
- OLED displays shall show stopped when tracking is stopped.
- OLED displays shall show tracking during normal on-source tracking.
- OLED displays shall show slewing only during gross movement between targets.
- OLED displays shall show scan on the selected scan antenna while preserving the real source name.
- OLED displays shall not show scan/Y factor state on the non-selected antenna.
- OLED displays shall clear or change to offline appropriately when the controller is not connected.

GUI Requirements

- The GUI shall be compact and readable on the target Ubuntu display.
- The GUI shall open at a useful default size without manual resizing.
- All menu buttons shall be visible at startup.
- GUI values shall use fixed-width fields sized for worst-case values.
- Live value changes shall not resize panes or clip neighbouring fields.
- Text shall never be truncated or overwritten.
- The GUI shall use colour to distinguish normal, active, warning, and fault states.
- Pane titles and labels shall be consistent in size, bolding, and capitalisation.
- Manual controls shall be compact and arranged intuitively.
- Status text shall not duplicate information already shown elsewhere.
- Stale status and stale measurement values shall be cleared when no longer valid.

Logging Requirements

- The event log shall record significant operational and fault events.
- The log shall avoid excessive routine verbosity.
- Log retention days shall be configurable.
- Scan, Y factor, and power measurement logs shall not clutter the application root directory.
- Logs shall contain enough information to diagnose unexpected stops, no motion, communication faults, and scan/Y factor failures.

Development And Maintenance Requirements

- WT7 shall keep the code modular enough for future GUI changes.
- WT7 shall retain the Tkinter legacy GUI only as a transition reference until PyQt5 dialogs are complete.
- Tests shall cover safety, configuration, tracking state, B210 power primitives, and GUI state behaviour.
- The application should eventually support a cleaner architecture separating GUI presentation from station-control services.
- GitHub repositories and local directories should be kept tidy as versions are superseded.

Current Implementation Notes

WT7 currently contains:

- PyQt5 main GUI in `wt7_ubuntu_gui.py`.
- Legacy Tkinter implementation retained in `wt7_tk_legacy_gui.py`.
- Serial/controller/safety logic in `wt7_antenna.py`.
- Configuration logic in `wt7_config.py`.
- B210 power logic in `wt7_b210_power.py`.
- Astronomy functions in `wt7_astro.py` and `wt7_solar.py`.
- Event logging in `wt7_logging.py`.
- Regression tests under `tests/`.

Some workflows are still evolving, especially the PyQt5 versions of secondary dialogs and the B210-powered calibration/Y factor paths. The current document is therefore both a description of the app and a requirements baseline for the next round of development.